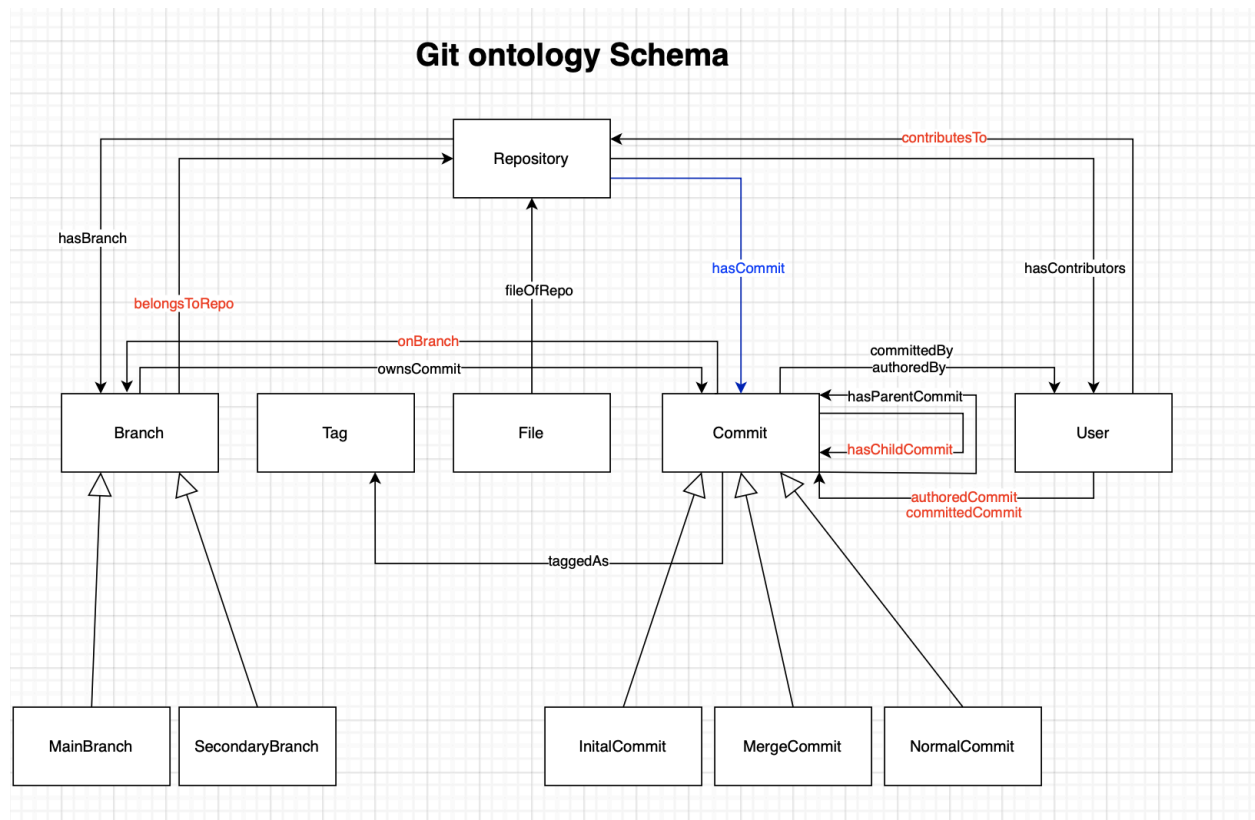


INDIVIDUAL REPORT

Andoni Karafilakis 24314657

Partner: Kamleshkumar Senthilkumar 24245674

Design Choices:



Schema + Ontology Design:

In beginning to create the schema for our ontology, we considered what sort of relationships and entities we would need to capture in our design. From the task sheet, we used the entities repository, file, user, branch and commit. From there, we realised we would also need to include the subclasses for Branch and Commit.

For Branch, we made the MainBranch and SecondaryBranch subclasses as seen in the above UML diagram. For Commit, we created the subclasses of InitialCommit (a commit with no parents), MergeCommit (a commit with 2 or more parents) and NormalCommit (a commit with

exactly 1 parent. Once we settled on the classes, the relations between them came quite naturally.

Initially we had an inverse relationship for almost every relation but realised that would not be necessary for the useful querying over the knowledge graph. In the UML diagram, the inverse relations are highlighted in red.

We also elected to include tags as a class as they are a key part of git that would need to be included in order to capture the full depth of the domain. In order to do this, we modified the original `git_data.py` file to use the Github REST API in order to capture the tags of a given commit and store them as a list in the `commits.json` file. We introduced a `taggedAs` relation to indicate when a commit is “tagged as” something, usually a release version (e.g. V1.1.2).

One of the problems we faced in our design of the application is that we were initially using equivalence relations in Protege rather than SWRL rules. We ran into many bugs and errors so quickly switched to SWRL rules using the `swrlb()` builtin which solved a lot of our problems. We were also initially using the hermit reasoner which no longer worked with the change to SWRL so we made the decision to reason using the Pellet Reasoner.

It is important to note that we designed the ontology using Protege. We made the choice to do so as its intuitive graphical interface made it much easier to add classes, relations, and check for inconsistencies than using a traditional script approach. The ability to visualise the relations had a big impact on my ability to understand the context of the ontology.

Web Application:

In designing the Flask application, we elected to go for a very simple design as to prioritise the functionality of the querying over the aesthetics and finished look of the website. It is a very simple design where hyperlinks are used to navigate between different sections, simply and efficiently.

We elected to use a flask application over a command line application as it is something we had more experience with from CITS3403 and would also allow us to better visualise the data using tables and a nicely spaced layout rather than with a command line interface that would be less natural to use.

Data Extraction:

As I already mentioned, we modified the `git_data.py` script to extract additional data from our sample repos. We also were very intentional in choosing our sample repositories all from the same organisation, `pytest-dev`, a framework for writing small python tests. This ensured that we were able to conduct useful and interesting queries to gain insight into how users in an organisation collaborate with each other using git.

This was most evident in the fact that our choice of repos allowed us to see the same users (and bots) contributing to a range of different repos within the same organisation.

When choosing which repositories to process, we searched the organisation in order of smallest to largest repos in order to prevent the populate and reason script from timing out/exceeding

memory limits. Once we had the smallest repositories, we then added some larger ones to make the queries more interesting whilst still allowing the ontology to be populated and reasoned. This design choice was fundamental in the success of our project.

Tools:

Protege was used to design the ontology as stated above as a simpler and more intuitive way to visualise the classes and relations. We also used the inbuilt reasoner functionality which was incredibly helpful in checking for logical inconsistencies. We also used it to add SWRL rules very easily. I would definitely recommend Protege as a tool for others to not only simplify the process, but also help them gain a better understanding of the schema they are developing through the GUI.

OWLReady2 was used in the python script responsible for populating and reasoning. It was helpful because it provides a direct way to work with the ontology using python objects. This made it very easy to automate the knowledge graph population. It also integrates well with the Pellet reasoner that we used which allows for the inference to happen also in python.

RDFLib was used for handling and manipulating the RDF data. It allowed us to query the knowledge graph effectively using SPARQL. It was very helpful because the SPARQL queries were able to integrate very nicely into our web application. Although I struggled initially with getting the hang of the SPARQL syntax, I would definitely say that it was the right choice to use for this style of project and I would recommend it.

Github REST API was used to pull real repository data fast and reliably. It worked consistently once I applied a GITHUB TOKEN to overcome the rate limit and was the base on which the whole project was built. The documentation was also relatively easy to understand and was helpful.

Flask was used to create the web app. It let us create endpoints for different data and trigger the necessary SPARQL queries whilst providing a lightweight way to visualise the results which was the important part. Flask allowed us to return rendered tables of information spread across multiple pages rather than a large list of incomprehensible data. At times it was running very slowly which was frustrating but I would still recommend it as I don't know of any alternatives that would be any quicker given the size of the data we gave it.

AI Assistance was used in the form of LLMs (mostly ChatGPT but also Claude). It was mostly used as a "web browser" for specific questions. It was also used to modify code in `git_data.py` to fetch different data and fix bugs that weren't in the original script. We also used AI to help generate html code and flask routes to assist the functional aspects (queries) of our website so we could focus more on the knowledge representation and less on the web development. We used it to search for solutions to obscure bugs/problems that we encountered plenty of throughout working on the project. AI Assistance was helpful in refactoring our code and adding logging/comments when we were working on bugfixes and struggling to find the source.

Time Estimates

These are of course very rough estimates as we worked on the project over a number of weeks.

Drafting The Schema: 8 Hours

Extracting Data from Repos/Modifying git_data.py: 4 Hours

Populating and Reasoning Script: 15 Hours

Creating Web App: 20 Hours

Trying to fix "Ontology Inconsistent" bugs: 13 hours

User Manual: 5 Hours

Individual Report: 3 Hours

Teamwork:

Work was divided in a way that we mostly both worked on the same task at the same time in collaboration rather than allocating entire tasks. This way we were both involved in all aspects of the project. That being said, I'd say that Kamleshkumar specialised more in the designing of the ontology and I was more involved in the creation of the web application/SPARQL queries. We collaborated well using a github repository, met up regularly face to face to work on the project as well as through remote meetings and I think the collaboration was very effective. I think that all in all, we probably both contributed similar amounts to the project and am very pleased with our partnership. It was great to work with Kamleshkumar!